

LAPORAN PRAKTIKUM
STRUKTUR DATA
PEKAN KE 9

Disusun Oleh :

Muhammad Irfan 2511531008

Dosen Pengampu : Dr Wahyudi S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2025

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum dari mata kuliah Struktur Data dengan judul binary tree, traversal tree dan traversal graph dengan baik dengan benar

Laporan ini disusun sebagai bentuk hasil kegiatan praktikum Struktur Data yang di mana membahas tentang bahasa pemrograman java dan penggunaan binary tree, traversal tree dan traversal graph serta kegunaan masing-masing tersebut serta cara penerapannya.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna baik dari segi penulisannya dan dari segi pembahasannya. Oleh karena itu saya ingin mengucapkan terima kasih kepada dosen pengampu, asisten praktikum yang telah membimbing serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, penulis ucapkan terima kasih pada dosen pengampu yang telah memberikan arahan dan semoga laporan ini dapat bermanfaat baik bagi penulis maupun pembaca dalam pemahaman binary tree, traversal tree dan traversal graph

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
1.Class Node_2511531008	2
Kode program	2
2. Class BTree_2511531008	4
Kode program	4
3.Class BTreeDriver_2511531008	6
Kode program	6
Ouput.....	8
4. GraphTraversal_2511531008	8
kodeprogram	8
OUPUT	10
BAB III PENUTUP	11
3.1 Kesimpulan	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen yang paling penting untuk menyimpan nilai dan mengelola nilai secara baik dan efisien. Pengurutan nilai dan data sangatlah penting dalam dunia pemrograman. Pengurutan dalam dunia pemrograman berfungsi untuk mengurutkan nilai agar mudah di baca, analisis dan dikelola oleh user

Salah satu struktur data yang banyak digunakan yakni Tree dan Graph. Tree merupakan struktur data hierarki yang terdiri dari node node yang saling terhubung membentuk sebuah pohon. Pada binary tree terdapat beberapa bentuk penelusuran traversal, seperti inorder, preorder dan postorder

Selain tree, graph merupakan bentuk struktur data yang saling mempresentasikan hubungan antar node yang lebih kompleks. penelusuran graph tersebut dapat dilakukan dengan cara algoritma BFS (breadth first search) dan DFS (Deep first search). DFS merupakan penelusuran secara mendalam, sedangkan BFS merupakan penelusuran dengan melebar berdasarkan level node. Diharapkan pada praktikum ini mahasiswa dapat memahami graph dan binary tree serta implementasi dalam kehidupan sehari hari

1.2 Tujuan Praktikum

Tujuan utama dari praktikum ini adalah memberikan pemahaman kepada mahasiswa tentang konsep binary tree dan graph serta implementasi di dalam pemrograman bahasa Java. Selain itu bagaimana pengaplikasian berbagai berbagai-macam problem dengan menggunakan binary tree dan graph

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah mahasiswa diharapkan dapat memahami konsep, binary tree dan graph Selain itu, mahasiswa juga diharapkan untuk dapat mengolah dan meningkatkan analisis terhadap problem dengan menggunakan algoritma traversal pada binary dan graph tersebut..

BAB II

PEMBAHASAN

1.Class Node_2511531008

Kode program

```
4 public class Node_2511531008 {
5     int data_1008;
6     Node_2511531008 left_1008;
7     Node_2511531008 right_1008;
8     public Node_2511531008(int data_1008) {
9         this.data_1008 = data_1008;
10        left_1008 = null;
11        right_1008 = null;
12    }
13    public void setLeft_1008(Node_2511531008 node_1008) {
14        if (left_1008 == null) left_1008 = node_1008;
15    }
16    public void setRight_1008(Node_2511531008 node_1008) {
17        if (right_1008 == null) right_1008 = node_1008;
18    }
19    public Node_2511531008 getLeft_1008() {
20        return left_1008;
21    }
22    public Node_2511531008 getRight_1008() {
23        return right_1008;
24    }
25    public int getdata_1008() {
26        return data_1008;
27    }
28    public void setdata_1008(int data_1008) {
29        this.data_1008 = data_1008;
30    }
31 }
```

Penjelasan :

- Membuat variabel data_1008, left_1008 dan right_1008.
- Membuat konstruktor dengan attribute berupa data, left_1008 dan right_1008.
- Program ini terdiri dari setter dan getter dari attribute left_1008, right_1008 serta data_1008

```

32 void printPreorder_1008(Node_2511531008 node_1008) {
33     if (node_1008 == null) return;
34     System.out.print(node_1008.data_1008+ " ");
35     printPreorder_1008(node_1008.left_1008);
36     printPreorder_1008(node_1008.right_1008);
37 }
38 void printPostorder_1008(Node_2511531008 node_1008) {
39     if (node_1008 == null) return;
40     printPostorder_1008(node_1008.left_1008);
41     printPostorder_1008(node_1008.right_1008);
42     System.out.print(node_1008.data_1008+ " ");
43 }
44 void printInorder_1008(Node_2511531008 node_1008) {
45     if (node_1008 == null) return;
46     printInorder_1008(node_1008.left_1008);
47     System.out.print(node_1008.data_1008+ " ");
48     printInorder_1008(node_1008.right_1008);
49 }

```

Penjelasan

- Ketiga method ini adalah bentuk dari penjelajah graf traversal
- Method print_preorder_1008. Jika node_1008 kosong, maka selesai. Jika tidak, Cetak dulu nilai data, kemudian lanjut menjelajah node kiri , selanjutnya node kanan
- Method print_postorder_1008. Jika node_1008 kosong, maka selesai. Jika tidak, Menjelajah dulu node kiri, kemudian lanjut menjelajah node kanan , selanjutnya cetak node
- Method print_inorder_1008. Jika node_1008 kosong, maka selesai. Jika tidak ,Menjelajah dulu node kiri , selanjutnya cetak node dan kemudian menjelajah node kanan

```

public String print_1008() {
    return this.print_1008("",true,"");
}

public String print_1008(String prefix_1008, boolean isTail_1008,String sb_1008) {
    if (right_1008 != null) {
        right_1008.print_1008(prefix_1008 + (isTail_1008 ? "| " : " "), false, sb_1008);
    }
    System.out.println(prefix_1008 + (isTail_1008 ? "\\-- " : "/-- ") + data_1008);
    if(left_1008 != null) {
        left_1008.print_1008(prefix_1008 + (isTail_1008 ? " " : "| "), true, sb_1008);
    }
    return sb_1008;
}

```

Penjelasan:

- Print_1008 memanggil method rekursif di bawahnya untuk menampilkan struktur pohon

- Print_1008 memiliki 3 parameter yakni prefix_1008, isTail_1008 dan sb_1008
- Method ini mencetak nilai node dalam bentuk tree. Jika node kanan masih tidak kosong, cetak anak node kanan. kemudian cetak nilai data. Kemudian , Jika node kiri tidak kosong, cetak anak node kiri

Ouput:

```
<terminated> ShellSort_2511531008 [Java Application] C:\Use
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2. Class BTree_2511531008

Kode program

```
1 package pekan9_2511531008;
2
3 public class BTree_2511531008 {
4     private Node_2511531008 root_1008;
5     private Node_2511531008 currentNode_1008;
6     public BTree_2511531008() {
7         root_1008 = null;
8     }
9     public boolean search(int data_1008) {
10         return search(root_1008, data_1008);
11     }
12     private boolean search(Node_2511531008 node, int data_1008) {
13         if (node == null) return false;
14
15         if (node.getdata_1008() == data_1008)
16             return true;
17         if (node.getLeft_1008() != null)
18             if (search(node.getLeft_1008(), data_1008))
19                 return true;
20         if (node.getRight_1008() != null)
21             if (search (node.getRight_1008(), data_1008))
22                 return true;
23         return false;
```

Penjelasan :

- program ini merupakan implementasi Tree dengan menggunakan node yang sudah dibuat sebelumnya
- terdapat 2 node, node pertama bernama root_1008 dan node kedua bernama currentNode_1008
- method search merupakan method untuk penelusuran sebuah node. Jika kosong return false. Jika ada kembalikan true
- lakukan juga penelusuran node kiri dan node kanan jika tidak bernilai kosong

```

25 public void printInorder_1008() {
26     root_1008.printInorder_1008(root_1008);
27 }
28 public void printPreOrder_1008() {
29     root_1008.printPreorder_1008(root_1008);
30 }
31 public void printPostOrder_1008() {
32     root_1008.printPostorder_1008(root_1008);
33 }

```

Penjelasan :

- Ketiga method ini memanggil root_1008 pada masing masing order. Misalkan Memanggil method Inorder pada root. Maka urutannya left -> root -> right

```

35 public Node_2511531008 getroot_1008() {
36     return root_1008;
37 }
38 public boolean isEmpty_1008() {
39     return root_1008 == null;
40 }
41 public int countNodes_1008(){
42     return countNodes_1008(root_1008);
43 }
44 private int countNodes_1008(Node_2511531008 node_1008) {
45     int count = 1;
46     if (node_1008 == null) {
47         return 0;
48     } else {
49         count += countNodes_1008 (node_1008.getLeft_1008());
50         count += countNodes_1008 (node_1008.getRight_1008());
51         return count;
52     }
53 }

```

Penjelasan :

- Terdapat getter root yang berfungsi sebagai mengambil nilai root_1008 dengan cara return
- Terdapat function isEmpty_1008 berfungsi sebagai mengecek apakah root bernilai null atau tidak
- Fungsi countNodes_1008 untuk menghitung banyak nodes melalui root_1008

```

55 public void print_1008() {
56     root_1008.print_1008();
57 }
58 public Node_2511531008 getCurrent_1008() {
59     return currentNode_1008;
60 }
61
62 public void setCurrent_1008(Node_2511531008 node) {
63     this.currentNode_1008 = node;
64 }
65
66 public void setroot_1008(Node_2511531008 root_1008) {
67     this.root_1008 = root_1008;
68 }
69 }

```

Penjelasan

- Terdapat fungsi print memanggil node root_1008
- Memiliki getter dan setter tiap node current_1008 dan root_1008

3.Class BTreeDriver_2511531008

Kode program

```

1 package pekan9_2511531008;
2
3 public class BTreeDriver_2511531008 {
4     public static void main(String[] args) {
5         // membuat pohon
6         BTree_2511531008 tree_1008 = new BTree_2511531008();
7         System.out.print("jumlah simpul awal pohon: ");
8         System.out.println(tree_1008.countNodes_1008());
9         //menambahkan simpul data
10        Node_2511531008 root_1008 = new Node_2511531008(1);
11        //menjadikan simpul 1 sebagai root
12        tree_1008.setroot_1008(root_1008);
13        System.out.println("Jumlah simpul jika hanya ada root");
14        System.out.println(tree_1008.countNodes_1008());
15        Node_2511531008 node2_1008 = new Node_2511531008(2);
16        Node_2511531008 node3_1008 = new Node_2511531008(3);
17        Node_2511531008 node4_1008 = new Node_2511531008(4);
18        Node_2511531008 node5_1008 = new Node_2511531008(5);
19        Node_2511531008 node6_1008 = new Node_2511531008(6);
20        Node_2511531008 node7_1008 = new Node_2511531008(7);
21        Node_2511531008 node8_1008 = new Node_2511531008(8);
22        Node_2511531008 node9_1008 = new Node_2511531008(9);
23    }
24 }

```

Penjelasan:

- Di awali dengan membuat tree dengan nama tree_1008
- Kemudian menghitung jumlah simpul pada pohon
- Menambahkan 1 simpul data dan sekaligus menjadikan root

- Baris 15-22 delarasi objek node baru

```

24     root_1008.setLeft_1008(node2_1008);
25     node2_1008.setLeft_1008(node4_1008);
26     node2_1008.setRight_1008(node5_1008);
27     node4_1008.setRight_1008(node8_1008);
28     root_1008.setRight_1008(node3_1008);
29     node3_1008.setLeft_1008(node6_1008);
30     node3_1008.setRight_1008(node7_1008);
31     node6_1008.setLeft_1008(node9_1008);
32     //set root
33     tree_1008.setCurrent_1008(tree_1008.getroot_1008());
34     System.out.println("menampilkan simpul terakhir");
35     System.out.println(tree_1008.getCurrent_1008().getdata_1008());
36     System.out.println("jumlah simpul: setelah simpul ? ditambahkan");
37     System.out.println(tree_1008.countNodes_1008());
38     System.out.println("InOrder: ");
39     tree_1008.printInorder_1008();
40     System.out.println("\nPreOrder");
41     tree_1008.printPreOrder_1008();
42     System.out.println("\nPostOrder");
43     tree_1008.printPostOrder_1008();
44     System.out.println(" \n Menampilkan simpul dalam bentuk pohon");
45     tree_1008.print_1008();

```

- Baris 24-31 merupakan perintah kode untuk menyusun struktur tree
- Baris 36-37 menghitung jumlah semua node
- Baris 38-43 menampilkan traversal inorder,preorder dan postorder
- Baris 44-45 mencetak struktur tree

Ouput

```
jumlah simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir
1
jumlah simpul: setelah simpul ? ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder
1 2 4 8 5 3 6 9 7
PostOrder
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|      /-- 7
|      /-- 3
|      |      \-- 6
|      |      \-- 9
|-- 1
|      /-- 5
|-- 2
|      |      /-- 8
|      |      \-- 4
```

4. GraphTraversal_2511531008

kodeprogram

```
3 public class GraphTraversal_2511531008 {
4
5     private Map<String,List<String>> graph_1008 = new HashMap<>();
6     // Menambahkan edge (graf tak berarah)
7
8     public void addEdge(String node1_1008, String node2_1008) {
9         graph_1008.putIfAbsent(node1_1008, new ArrayList<>());
10        graph_1008.putIfAbsent(node2_1008, new ArrayList<>());
11        graph_1008.get(node1_1008).add(node2_1008);
12        graph_1008.get(node2_1008).add(node1_1008);
13
14    }
15    //menampilkan graf awal
16    public void printgraph_1008() {
17        System.out.println("Graf Awal (Adjacency List) :");
18        for (String node_1008 : graph_1008.keySet()) {
19            System.out.print(node_1008 + " -> ");
20            List<String> neighbors_1008 = graph_1008.get(node_1008);
21            System.out.println(String.join(", ",neighbors_1008));
22        }
23    }
```

Penjelasan :

- Diawali dengan membuat graph dengan nama graph_1008

- Baris 8 -12 menginputkan nilai pada masing masing node.masing – masing node di arahkan ke node tertentu
- Baris 16-21 menampilkan graph awal dalam bentuk adjacency list

```

24      //dfs rekursif
25      public void dfs_1008(String start_1008) {
26          Set<String> visited_1008 = new HashSet<>();
27          System.out.println("Penelusuran DFS:");
28          dfsHelper_1008(start_1008,visited_1008);
29          System.out.println();
30      }
31      private void dfsHelper_1008(String current_1008,Set<String> visited_1008) {
32          if (visited_1008.contains(current_1008)) return ;
33          visited_1008.add(current_1008);
34          System.out.print(current_1008 + " ");
35          for(String neighbor : graph_1008.getDefault(current_1008, new ArrayList<>())) {
36              dfsHelper_1008(neighbor,visited_1008);
37          }
38      }

```

Penjelasan:

- Method ini melakukan penjelajahan graf dengan metode dfs (depth first search)
- Menggunakan set<string> visited untuk menandai node yang sudah dikunjungi
- Menggunakan stack dan rekursi sebagai bentuk penjelajahan graf traversal

```

40      public void bfs_1008(String start_1008) {
41          Set<String> visited_1008 = new HashSet<>();
42          Queue<String> queue_1008 = new LinkedList<>();
43          queue_1008.add(start_1008);
44          visited_1008.add(start_1008);
45          System.out.println("penelusuran BFS:");
46          while(!queue_1008.isEmpty()) {
47              String current_1008 = queue_1008.poll();
48              System.out.print(current_1008 + " ");
49              for(String neighbor : graph_1008.getDefault(current_1008, new ArrayList<>()))
50                  if(!visited_1008.contains(neighbor)) {
51                      queue_1008.add(neighbor);
52                      visited_1008.add(neighbor);
53                  }
54          }
55          System.out.println();

```

Penjelasan :

- Method ini melakukan penjelajahan graf dengan metode bfs (breadth first search)
- Menggunakan iteratif dan Queue sebagai bentuk penjelajahan graf traversal

```

58 public static void main(String[] args) {
59     GraphTraversal_2511531008 graph_1008 = new GraphTraversal_2511531008();
60     //contoh graf : A-B, A-C, B-D, B-E
61     graph_1008.addEdge("A", "B");
62     graph_1008.addEdge("A", "C");
63     graph_1008.addEdge("B", "D");
64     graph_1008.addEdge("B", "E");
65     //cetak graf awal
66     System.out.println("graf awal adalah: ");
67     graph_1008.printgraph_1008();
68     //lakukan penelusuran
69     graph_1008.dfs_1008("A");
70     graph_1008.bfs_1008("A");
71 }
72

```

Penjelasan :

- Program utama ini membuat objek graph
- Setelah objek di buat, maka nilai graf diisi, kemudian cetak nilai graph
- Terdapat juga ouput penelusuran graf dengan cara DFS dan BFS

OUPUT

```

graf awal adalah:
Graf Awal (Adjacency List) :
A - > B, C
B - > A, D, E
C - > A
D - > B
E - > B
Penelusuran DFS:
A B D E C
penelusuran BFS:
A B C D E

```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum ini, dapat disimpulkan bahwa binary tree dan graph dapat diimplementasikan dengan baik dengan menggunakan bahasa pemrograman java. Binary tree dapat dilakukan secara traversal seperti inorder, preorder dan postorder untuk menhunjungi node dengan urutan yang tertentu. Selain itu, Graph dipresentasikan dalam bentuk adjacency list dalam menghubungkan antara node satu dengan node yang lain. Penelusuran graph dapat dilakukan dengan algoritma BFS dan DFS. Melalui praktikum ini, mahasiswa dapat memahami lebih baik mengenai konsep binary tree dan graph serta implementasi dalam pengembangan aplikasi dan pemecahan masalah